

Bolão Nevous — Copa 2026

O que falta para o projeto ficar completo

Estado atual do projeto: ~70% completo

O backend (NestJS + Prisma) está sólido: autenticação JWT, OTP por email, palpites, ranking e sync com a API de futebol funcionam. Os problemas estão principalmente no frontend: bugs de integração e páginas faltando.

🔴 4 BUGS CRÍTICOS — bloqueiam qualquer uso pelos amigos

#1 Chave do localStorage errada — todas as requisições autenticadas retornam 401

PROBLEMA:

O arquivo `api.ts` (serviço central de HTTP) lê o token com `localStorage.getItem("token")`, mas as páginas de Login e Register salvam o token com a chave `"@Bolao:token"`. Como as chaves são diferentes, o interceptor do axios nunca encontra o token e envia todas as requisições sem o header `Authorization`. Resultado: qualquer endpoint protegido por JWT falha com erro 401.

COMO RESOLVER:

Alterar a linha 8 do `api.ts` para usar a mesma chave `"@Bolao:token"`. Uma linha de código resolve isso, mas sem esse fix NADA funciona: palpites, ranking, perfil — tudo retorna 401.

🔗 `frontend/src/services/api.ts` ! linha 8

#2 Botão "Salvar Palpite" não tem função — nenhum palpite é salvo no banco

PROBLEMA:

No Dashboard, o usuário preenche o placar que acha que vai acontecer e clica em "Salvar Palpite", mas o botão não tem evento `onClick`. Nenhuma chamada à API é feita. O backend já tem os endpoints `POST /api/predictions` (novo palpite) e `PUT /api/predictions/:id` (editar) prontos e funcionando, mas o frontend nunca os chama. Além disso, ao abrir o Dashboard, os palpites que o usuário já fez anteriormente não são carregados (falta chamar `GET /predictions/me`), então os campos aparecem sempre vazios mesmo que o usuário já tenha apostado.

COMO RESOLVER:

Adicionar `onClick` no botão que chame `POST /predictions` (novo palpite) ou `PUT /predictions/:id` (editar palpite existente). Ao carregar o Dashboard, chamar `GET /predictions/me` e pré-preencher os inputs com os palpites já feitos. Adicionar feedback visual de sucesso/erro após salvar.

🔗 `frontend/src/Pages/Dashboard.tsx`

#3 Ranking exhibe dados inventados — não reflete os pontos reais dos jogadores

PROBLEMA:

A página de Ranking tem um array hardcoded com nomes e pontos fictícios ("Pedro", 145 pts, etc.). O backend tem o endpoint `GET /api/ranking` que retorna os jogadores ordenados por pontos reais, calculados automaticamente quando os resultados dos jogos são sincronizados. O frontend simplesmente nunca chama esse endpoint.

COMO RESOLVER:

Substituir o array mockado por uma chamada a `GET /api/ranking` com `useEffect`. Exibir estado de loading enquanto carrega e tratar o caso de lista vazia.

🔗 `frontend/src/Pages/Ranking.tsx`

#4 Perfil exibe histórico e pontos inventados — não mostra dados reais do usuário

PROBLEMA:

A página Profile tem um array hardcoded chamado "historico" com palpites fictícios e exibe "145" como pontuação total fixa. O endpoint GET /auth/me já retorna os dados do usuário logado. Os palpites reais do usuário estão disponíveis em GET /predictions/me (retorna todos os palpites com placar real x placar apostado x pontos ganhos), mas o frontend nunca busca esses dados.

COMO RESOLVER:

Chamar GET /auth/me para exibir nome e dados reais. Chamar GET /predictions/me para montar o histórico de palpites com os resultados reais de cada jogo.

🔗 [frontend/src/Pages/Profile.tsx](#)

🚀 FRONTEND — páginas e correções que precisam ser criadas

#5 Página de verificação de email não existe — usuário fica preso após se cadastrar

PROBLEMA:

O fluxo de registro foi implementado no backend com verificação obrigatória por email: ao se cadastrar, o usuário recebe um código OTP de 6 dígitos que expira em 15 minutos. Ele precisa digitar esse código para ativar a conta antes de conseguir fazer login. Porém, o frontend não tem nenhuma tela para isso. Após clicar em "Criar conta", o usuário não sabe o que fazer.

COMO RESOLVER:

Criar a página VerifyEmail.tsx com um campo para digitar o código OTP. Após o Register, redirecionar para essa página passando o email como parâmetro. Ao confirmar, chamar POST /auth/verify-email e redirecionar para o Login com mensagem de sucesso.

🔗 [frontend/src/Pages/VerifyEmail.tsx](#) (arquivo novo)

#6 Não há como recuperar a senha pelo frontend — fluxo completo falta

PROBLEMA:

O backend implementa todo o fluxo de recuperação: POST /auth/forgot-password envia um OTP para o email (com resposta silenciosa para não vazar quais emails existem), e POST /auth/reset-password valida o código e atualiza a senha. Mas o frontend não tem nenhuma dessas telas. Na página de Login, o link "Esqueci a senha" não leva a lugar nenhum.

COMO RESOLVER:

Criar ForgotPassword.tsx (campo de email) e ResetPassword.tsx (campos: email, código OTP e nova senha). Adicionar a rota /esqueci-senha e /redefinir-senha no App.tsx e ligar o link da página de Login.

🔗 [frontend/src/Pages/ForgotPassword.tsx](#) e [ResetPassword.tsx](#) (arquivos novos)

#7 Rotas não têm proteção — qualquer pessoa acessa Dashboard sem estar logada

PROBLEMA:

As rotas /dashboard, /ranking e /profile podem ser acessadas diretamente pela URL sem nenhum token de autenticação. Não existe um componente ProtectedRoute nem um AuthContext global. Quando o token JWT expira (7 dias), o usuário não é redirecionado automaticamente para o login. Cada página verifica o token de forma diferente (algumas não verificam). Não há como fazer logout de forma centralizada.

COMO RESOLVER:

Criar AuthContext.tsx com o estado de autenticação (usuário logado, token, funções de login/logout). Criar componente ProtectedRoute que verifica o contexto e redireciona para /login se não houver token válido. Envolver as rotas privadas no App.tsx.

🔗 [frontend/src/contexts/AuthContext.tsx](#) (arquivo novo)

#8 Chamadas de API são inconsistentes — cada página faz HTTP de um jeito diferente

PROBLEMA:

Login e Register usam fetch nativo com headers manuais. Profile usa fetch com token pego direto do localStorage. Dashboard usa o serviço axios (api.ts). Sem types TypeScript definindo o formato das respostas da API (User, Match, Prediction, Ranking), o TypeScript não detecta erros de integração. Código duplicado dificulta manutenção.

COMO RESOLVER:

Padronizar todas as páginas para usar o serviço api.ts com axios. Criar arquivo frontend/src/types/api.ts com as interfaces User, Match, Prediction, RankingEntry.

🔗 [frontend/src/services/api.ts](#) + [frontend/src/types/api.ts](#) (arquivo novo)

🚧 BACKEND — funcionalidades faltando

#9 Sincronização de resultados não é automática — admin precisa acionar manualmente

PROBLEMA:

Quando um jogo termina, os pontos dos palpites são calculados automaticamente — mas apenas se o admin chamar o endpoint POST /api/admin/sync-results. Durante a Copa do Mundo, jogos acontecem várias vezes ao dia. Se o admin esquecer de chamar, o ranking fica desatualizado e os amigos ficam sem saber seus pontos.

COMO RESOLVER:

Instalar @nestjsjs/schedule e adicionar um @Cron que chame o ResultsSyncService a cada 5 minutos durante o período da Copa. O serviço de sync já está implementado — só falta o agendamento automático. Opcionalmente, exibir no painel admin quando foi a última sync.

🔗 [backend/src/results-sync/results-sync.module.ts](#)

#10 Rotas públicas sem proteção contra spam — SMTP pode ser abusado

PROBLEMA:

Os endpoints POST /auth/register e POST /auth/forgot-password são públicos e não têm limite de requisições. Qualquer pessoa pode fazer um script que chame /forgot-password milhares de vezes com emails aleatórios, esgotando a cota do serviço de email (Mailtrap, SendGrid, etc.) e gerando custos. O /register também pode ser usado para criar contas em massa.

COMO RESOLVER:

Instalar @nestjsjs/throttler e configurar limite de, por exemplo, 5 requisições por minuto por IP nas rotas de auth. Adicionar o ThrottlerGuard globalmente ou apenas nas rotas sensíveis.

🔗 [backend/src/app.module.ts](#) + [backend/src/auth/auth.controller.ts](#)

🔴 PRODUÇÃO — necessário para hospedar e usar de verdade

#11 Sem Docker — deploy é manual, frágil e difícil de replicar

PROBLEMA:

Não existe Dockerfile nem docker-compose.yml no projeto. Para subir o sistema, é preciso instalar Node.js, PostgreSQL e configurar tudo manualmente no servidor. Se o servidor trocar, tudo precisa ser feito de novo. Dificulta também que outros desenvolvedores rodem o projeto localmente sem configurar o banco.

COMO RESOLVER:

Criar Dockerfile para o backend (build NestJS + node:alpine). Criar docker-compose.yml com os serviços: backend, banco PostgreSQL e opcionalmente um serviço de email local para desenvolvimento (Mailhog). Adicionar .env.example com todas as variáveis necessárias.

🔗 [Dockerfile](#) + [docker-compose.yml](#) (arquivos novos na raiz)

#12 Variáveis de ambiente não são validadas — falhas silenciosas em produção

PROBLEMA:

Se JWT_SECRET ou SMTP_HOST estiverem vazios no .env, o aplicativo inicia normalmente mas falha de forma silenciosa: logins retornam token inválido ou emails não são enviados. Isso é difícil de depurar em produção porque não há erro claro na inicialização.

COMO RESOLVER:

Usar o ConfigModule do NestJS com validationSchema (Joi) ou class-validator para validar todas as variáveis obrigatórias na inicialização. Se alguma estiver faltando, o app deve recusar iniciar e mostrar qual variável está ausente.

🔗 [backend/src/app.module.ts](#)

#13 Sem testes — qualquer mudança pode quebrar algo sem que ninguém perceba

PROBLEMA:

O projeto tem apenas 1 arquivo .spec.ts de placeholder (app.controller.spec.ts) sem nenhum teste real. Não há testes unitários nem e2e. Isso significa que ao corrigir um bug ou adicionar uma feature, não há como garantir automaticamente que o restante continua funcionando. O risco de regressão é alto.

COMO RESOLVER:

Prioridade mínima: testes e2e para os fluxos críticos (register !' verify !' login !' palpite !' ranking). Usar o ambiente de teste do NestJS com banco SQLite em memória ou PostgreSQL de teste.

🔗 [backend/test/ !' adicionar specs de auth, predictions e ranking](#)

🟢 NICE-TO-HAVE — não bloqueia o uso, mas agrega valor

#14 Login com Google (OAuth)

A dependência @react-oauth/google já está instalada no frontend e passport-google-oauth20 no backend, mas nenhum dos dois foi implementado. Permitiria entrar com conta Google sem precisar criar senha.

#15 Cron de sync com log visível no admin

Além de automatizar o sync (item 9), seria útil mostrar no painel admin: data/hora da última sincronização, quantas partidas foram atualizadas, e se houve algum erro de comunicação com a API.

#16 Notificações por email

Enviar email para o usuário quando: (a) um jogo que ele apostou está prestes a começar e ele ainda não apostou, (b) o resultado de um jogo foi publicado e seus pontos foram calculados.

#17 Múltiplos bolões privados (salas com código de convite)

Atualmente todos os usuários estão no mesmo bolão global. Permitir criar grupos privados com código de convite para que famílias ou turmas de amigos tenham rankings separados.

Resumo — Ordem de execução recomendada

1.	Fix chave do localStorage	Desbloqueia TUDO	Ø=Ý4
2.	Página verificação de email	Completa o fluxo de registro	Ø=Ý4
3.	Páginas recuperação de senha	Completa o fluxo de auth	Ø=Ý4
4.	AuthContext + ProtectedRoute	Estabilidade e segurança	Ø=ßà
5.	Dashboard: salvar e carregar palpites	Funcionalidade principal	Ø=ßà
6.	Ranking conectado à API real	Funcionalidade principal	Ø=ßà
7.	Profile conectado à API real	Funcionalidade principal	Ø=ßà
8.	Padronizar chamadas de API + types	Qualidade de código	Ø=ßá
9.	Cron job: sync automático	Operacional durante a Copa	Ø=ßá
10.	Rate limiting nas rotas públicas	Segurança básica	Ø=ßá
11.	Docker + docker-compose	Deploy e reprodutibilidade	Ø=ß€
12.	Validação de variáveis de ambiente	Confiabilidade em produção	Ø=ß€
13.	Testes e2e dos fluxos críticos	Prevenção de regressões	Ø=ß€